SFA – Governance and Team Model

Overview

SmallFarmsAgents operates under the **Agents OS (AOS)** governance framework as an L0 spoke project. AOS is a multi-agent orchestration methodology that structures how AI agents collaborate, validate each other's work, and maintain accountability in software development. This document explains SFA's team structure, the AOS governance model, and how the two work together.

The AOS Governance Framework

Agents OS (AOS) is a methodology and tooling system developed by Nimrod (Team 00) to enable reliable, accountable AI-assisted software development. AOS answers the question: how do you run a software project with multiple AI agents – across different AI models, different tools, different environments – and maintain quality, consistency, and governance?

AOS is not a product. It is infrastructure – the rules, processes, file conventions, and tooling that governs all projects under the MyFarmAgents umbrella.

The core governance principles:

- **Builder ≠ Validator:** The AI agent that builds a feature cannot validate it. Validation must be performed by a different agent – ideally a different AI model entirely. This prevents self-validation bias.
- **Specification before implementation:** Every feature requires a written specification (LOD format) before any code is written. This forces clarity and prevents ambiguity from being resolved silently in code.
- **Single source of truth:** PostgreSQL is the data authority. No parallel state in files or spreadsheets. API-only mutations when the database is online.
- **Artifact-based communication:** Teams communicate by writing files to their `_COMMUNICATION/team_XX/` directories. There are no chat channels, no informal handoffs – every inter-team decision is a file.

AOS Deployment Profiles

AOS defines three deployment profiles for spoke projects:

Profile	Name	Description
L0	Lean / Manual	Lightweight governance – team assignments, gate records, communication artifacts. No automated dashboard. For smaller, simpler, or volunteer projects.
L2	AOS Dashboard	Full governance engine – PostgreSQL + FastAPI backend + Dashboard UI. For complex, multi-team, long-running projects.
L2.5	Managed Pipeline	Extension of L2 for complex work packages requiring multi-phase managed execution with human checkpoints.

SFA runs at L0. This is the appropriate profile for a volunteer community project: governance rigor without the overhead of running a full AOS backend alongside the application. The L0 profile means:

- Gate records are maintained in `_aos/roadmap.yaml`
- Team communication uses the `_COMMUNICATION/` directory structure
- Lean Kit modules are physically copied (not symlinked) into `lean-kit/`
- No AOS API server running alongside the SFA pipeline
- The AOS hub (agents-os repository) handles hub-level governance; SFA uses read-only governance snapshots

---

SFA Team Structure

SFA has five specialized teams, each with a defined scope and authority:

Team ID	Name	Role	Engine
sfa_sd / Team 00	System Designer	Product owner, final authority on all decisions	Nimrod (human)
sfa_arch / Team 100	Architecture	System architect – specifications, gate decisions, roadmap, mandates	Claude Code
sfa_val / Team 90	Validation	QA and constitutional validator – gate validation, test sign-off	Cursor (cross-engine)
sfa_team_10 / Team 10	Feature Dev	Implements collectors, parsers, normalizer, aggregator, admin UI, publisher	Claude Code
sfa_team_20 / Team 20	Infrastructure	Database schema, Alembic migrations, models, seed data, environment setup	Claude Code
sfa_team_50 / Team 50	QA Validation	Quality gate validation – runs integration tests, data quality checks, issues PASS/FAIL	Claude Code
sfa_team_80 / Team 80	Product & Strategy	Product research, UX guidance, content, community engagement	OpenAI (online)
sfa_team_190 / Team 190	Constitutional Review	Package-level review of governance documents and LOD200+ specifications	OpenAI Codex

Team 61 is referenced in the roadmap as the operational team responsible for waldhomeserver monitoring. Team 61 communicates via the cross-host file protocol (waldhomeserver → Mac inbox).

Cross-Engine Validation: The Core Governance Principle

The most important governance rule in AOS – and SFA inherits this fully – is **Iron Rule #1: the builder engine cannot be the validator engine**.

In SFA's case:
- **Team 10 and Team 20** (builders) run on Claude Code

- **Team 90** (QA validator) runs on Cursor (a different AI model)
- **Team 190** (constitutional validator) runs on OpenAI Codex (a completely different vendor)

This is not bureaucracy for its own sake. It is a structural defense against a specific failure mode: AI agents that write code tend to validate that code against the same reasoning patterns that produced it. A different model, with different training and different tendencies, catches different classes of errors.

In SFA's M1-M9 development, every gate required Team 50 (QA) to independently run the test suite and validate acceptance criteria – separately from the implementing team. Gate G1 had 7 specific QA tests. Gate G2 required a live collection run. Gate G7 required Nimrod's personal sign-off.

The Gate Lifecycle

Every work package passes through a formal gate sequence:

Gate	Code	Owner	Purpose
Eligibility	L-GATE_E	Team 190 (SFA: Team 100)	Is this work package appropriate? Scope, risk, dependencies clear?
Specification	L-GATE_S	Team 190	Is the spec complete enough to build from?
Build	L-GATE_B	Team 90	Does the implementation meet the spec? Tests passing?
Validate	L-GATE_V	Team 90 (final: Team 190)	Full constitutional validation. Implementation correct and complete?

No team advances to the next milestone until the gate is formally signed. In SFA's development, this meant: G1 must PASS before M2 work begins, G2 must PASS before M3, and so on. This sequential discipline is what gave each milestone a clean foundation to build on.

For large governance packages (LOD200+), Team 190 is the package reviewer. SFA's Post-M9 direction document (`SFA_POST_M9_PRODUCT_DIRECTION_LOD200_v1.0.0.md`) was submitted to Team 190 for package review before execution mandates are issued.

LOD Standard: How Specifications Are Written

AOS uses the **LOD (Level of Development) Standard** for specifications. Every specification is written to a declared level:

Level	Name	Content
LOD100	Concept	What problem, why it matters, rough scope
LOD200	Requirements	What will be built, acceptance at package level, non-goals
LOD300	Design	How it will be built – data model, module structure, key decisions
LOD400	Implementation Spec	Detailed acceptance criteria, test plan, API contracts
LOD500	Locked	Implementation complete, documented, gate closed

SFA's M1–M9 development used LOD400 specifications for implementation mandates – detailed enough to pass a formal gate. The Post-M9 direction document is LOD200 (requirements + non-goals), appropriate for a planning document before detailed design begins.

The Lean Kit

The **Lean Kit** is AOS's modular methodology library – a set of process documents covering how to handle specific governance scenarios. SFA has a physical copy of the lean-kit snapshot in its `_aos/lean-kit/` directory.

Key modules present in SFA's lean-kit:

- `validation-quality/` – gate lifecycle, QA mandate templates, validate_aos.sh
- `managed-pipeline/` – L2.5 reference (not active for SFA, but present for reference)
- `project-governance/` – governance change request process
- `12-home-server-infrastructure/` – waldhomeserver deployment, port registry

The lean-kit is **physically copied** into each spoke project, not symlinked. This ensures that if the hub lean-kit evolves, spoke projects are not automatically changed – they adopt new lean-kit versions intentionally, after review.

---

validate_aos.sh: Automated Governance Validation

The `validate_aos.sh` script is a 26-check automated validation suite that verifies the AOS governance structure of a project is intact:

- `_aos/` directory structure complete and correctly formed
- `CLAUDE.md` present and conformant
- Team communication directories present
- Lean-kit snapshot present
- Hub registration consistent
- Port registry integrity (for projects with long-running services)
- Cross-project boundary rules (no spoke writing to hub space)
- No forbidden patterns in the codebase (per `project_identity.yaml`)

SFA current status: 26 PASS / 9 SKIP / 0 FAIL (at S001 closure, April 12, 2026)

The 9 SKIP checks are advisory checks for L2/L2.5 features that are not applicable to L0 projects. They are expected SKIPs, not failures.

Communication Protocol

All inter-team communication in SFA follows the AOS artifact-file protocol:

- Each team has a `_COMMUNICATION/TEAM_XX/` directory
- Task assignments are written as mandate files: `MANDATE_[SCOPE].md`
- QA reports are written as: `QA_MANDATE_GN.md` (gate-specific)
- Gate verdicts are written as: `GATE_VERDICT_GN.md`
- Architecture decisions are written as: `ARCH_DECISION_[TOPIC].md`

There are no informal channels. Every decision that affects the project record is a file. This creates a complete audit trail of what was decided, when, and by whom.

Nimrod (Team 00) acts as the message router – reading mandate files, copying or presenting them to the appropriate AI agent session, and collecting the response artifacts.

AOS Hub vs. SFA Spoke

SFA exists within the AOS governance hierarchy:

AOS Hub (agents-os repository):
- Canonical governance definitions
- Iron Rules (14 rules that apply to all spokes)
- Lean Kit source (spokes get physical snapshots)
- Hub-level team definitions
- Cross-project project registry

SFA Spoke:
- L0 governance snapshot in `_aos/`
- SFA-specific team assignments (`team_assignments.yaml`)
- SFA roadmap and gate history (`roadmap.yaml`)
- SFA-specific communication artifacts (`_COMMUNICATION/`)
- No authority to modify hub governance – any change to AOS-level rules requires a Governance Change Request (GCR) filed through Team 100

This hub-spoke model means SFA benefits from AOS governance improvements (new lean-kit modules, validation improvements) while remaining insulated from hub-level changes it hasn't explicitly adopted.

Governance Maturity: What AOS Gives SFA

For a volunteer community project, AOS governance might seem heavyweight. The actual benefit is specific:

Confidence in the data. SFA publishes a price index that communities and farmers rely on. Every stage of the pipeline – from collector to normalizer to aggregator to publisher – was built

against a formal specification and validated by an independent team. The `100%` resolution rate is not a claim; it is a verified gate result.

Maintainability. SFA has been developed over six weeks from zero. The codebase has `127` passing tests, `31` Alembic migrations, and complete documentation in `documentation/`. New contributors (AI or human) can onboard from the documentation hub without relying on oral knowledge.

Change safety. Every modification to the normalization catalog (aliases, scope-skip rules, unit conversions) is in the database – versioned, auditable, reversible without code changes. Every schema change is a migration. Nothing is silent.

Credibility with partners and funders. A community project with professional-grade governance documentation, a formal milestone history, a clear roadmap, and a validated codebase is a fundamentally different kind of project from a prototype. AOS gives SFA the documentation posture of a professional software team, not a weekend hack.